

Aula 4 - Arquivos e Pastas: Vendo mais a fundo

Na nossa jornada pelo Linux, já entendemos como a rede funciona e como acessar computadores remotamente. Agora que estamos dentro do sistema, precisamos aprender a manipular o seu "mundo físico" digital: os arquivos e diretórios. A organização de arquivos é uma das ferramentas mais poderosas para o desenvolvedor e, diferentemente de sistemas operacionais focados apenas em interfaces gráficas, o terminal nos dá um controle cirúrgico sobre cada bit armazenado.

No Linux, arquivos e diretórios não são apenas nomes em uma pasta. Cada um carrega consigo informações de propriedade, permissões de acesso, tipo e muito mais. Neste capítulo, vamos explorar na prática como filtrar arquivos com curingas, criar atalhos inteligentes usando links simbólicos, entender e modificar as permissões de acesso, além de transferir a propriedade de arquivos entre usuários e grupos.

1. Arquivos: A Anatomia de um Diretório

Ao longo desta aula, usamos o `ls -l` diversas vezes para verificar permissões, donos e grupos de arquivos. Mas essa saída carrega muito mais informação do que apenas isso. Agora que você já domina o sistema de permissões e propriedades, é o momento certo para dissecar cada coluna e entender o que o terminal está te dizendo de forma completa.

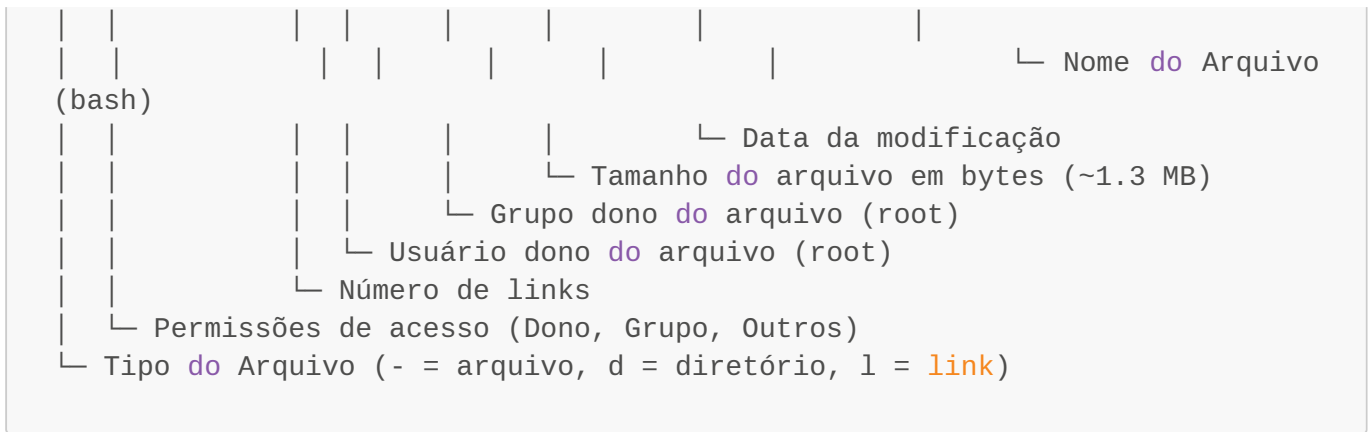
Para visualizar isso na prática, vamos listar o conteúdo da pasta `/bin`, que é onde ficam os programas essenciais do sistema:

```
(base) thallys@thallys-note:~$ ls -l /bin/
total 842320
-rwxr-xr-x 1 root  root      51648 fev  8  2024 '['
lrwxrwxrwx 1 root  root      24 ago  5  2021 aarch64-linux-gnu-gcc ->
aarch64-linux-gnu-gcc-11
-rwxr-xr-x 1 root  root    1298960 mai 30  2023 aarch64-linux-gnu-gcc-11
-rwxr-xr-x 1 root  root    1396520 mar 14  2024 bash
-rwsr-xr-x 1 root  root     44808 fev  6  2024 chsh
-rwxr-xr-x 1 root  root     141832 fev  8  2024 cp
-rwxr-xr-x 1 root  root   44015908 fev  2 14:17 docker
(etc...)
```

Ao executar `ls -l`, o terminal exibe uma tabela rica em detalhes. Primeiramente, observe a linha do topo: o sistema informa o total de blocos (`total 842320`) que o diretório inteiro ocupa no disco. Em computadores Linux padrão, cada bloco geralmente equivale a 1 kilobyte (`1024 bytes`). Portanto, em uma máquina de desenvolvimento completa, essa pasta facilmente ocupa mais de 800 megabytes.

Para que não reste nenhuma dúvida de como ler as informações dos arquivos logo abaixo, vamos pegar a linha referente ao programa `bash` (o próprio terminal que estamos usando) e dissecá-la. Pense no esquema a seguir como um mapa que revela onde cada pedaço de informação está posicionado:

```
-  rwxr-xr-x  1  root  root  1396520  mar 14 2024  bash
T  T         T  T     T         T         T
```



Você já reconhece boa parte desse mapa: as permissões, o usuário dono, o grupo e o tipo de arquivo são exatamente o que trabalhamos durante toda esta aula. Com esse contexto, a tabela abaixo fecha o ciclo e explica as colunas restantes:

Coluna na Saída	Significado e Estrutura	Exemplos Práticos
Tipo de Arquivo	O primeiro caractere indica a natureza do item.	- (arquivo comum) - d (diretório/pasta) - l (link/atalho) - b (dispositivo de bloco): Lida com dados em blocos, como HDs e pendrives (ex: /dev/sda). - c (dispositivo de caractere): Lida com dados byte a byte, como portas seriais e terminais (ex: /dev/tty).
Permissões	Nove caracteres (rwxr-xr-x) que definem regras de acesso.	- r (leitura) - w (escrita) - x (execução)
Propriedade	Exibe o nome do usuário dono e o grupo ao qual pertence.	- root (usuário e grupo do sistema) - thallys (seu usuário e grupo padrão) - Outros grupos: - docker (permissão para rodar contêineres)
Tamanho	Representa o peso do arquivo em bytes.	1396520 (aprox. 1.39 MB)
Data	Indica quando o arquivo foi criado ou modificado pela última vez.	mar 14 2024
Nome	O nome real dado ao arquivo, pasta ou link.	- Arquivo comum: bash - Se for link, mostra o destino: aarch64-linux-gnu-gcc -> aarch64-linux-gnu-gcc-11

2. Curingas no Linux (Wildcard)

Em um laboratório de robótica, é comum termos pastas lotadas com centenas de arquivos de log, códigos-fonte e imagens de sensores distribuídos em vários subdiretórios. Muitas vezes, você deseja visualizar, copiar ou apagar apenas um grupo específico de arquivos sem ter que fazer isso um por um. Para isso, utilizamos os Wildcards (caracteres curinga), que são símbolos especiais que substituem partes do nome de um arquivo durante uma busca, filtrando exatamente o que precisamos.

Na prática, os wildcards buscam **nomes de arquivos ou diretórios**. Eles são interpretados diretamente pelo próprio terminal (o shell) e funcionam em conjunto com os comandos de manipulação de arquivos (como `ls`, `cp`, `rm`, `mv` e `find`).

Aqui estão os wildcards mais importantes e como utilizá-los no dia a dia:

Curinga	Função Principal	Estrutura de Uso	Exemplos Práticos (Projeto Sonho/W2G)
*	Corresponde a 0 ou mais caracteres arbitrários de um nome de arquivo ou diretório.	• <code><comando> *</code>	• <code>ls *</code> : lista todas as pastas e arquivos na raiz do projeto.
		• <code><comando> <prefixo>*</code>	• <code>cp configs/robo* testes/</code> : copia todos os arquivos que começam com "robo" (ex: <code>roboA.cfg</code>).
		• <code><comando> *. <extensão></code>	• <code>rm logs/*.txt</code> : apaga todos os arquivos de texto da pasta logs (ex: <code>erro_motor.txt</code>).
		• <code><comando> * <palavra>*</code>	• <code>ls docs/*erro*</code> : lista arquivos com a palavra "erro" no nome (ex: <code>relatorio_erro.pdf</code>).
		• <code><comando> */*</code>	• <code>ls */*</code> : lista todos os arquivos contidos nas subpastas imediatas (<code>configs</code> , <code>docs</code> , <code>logs</code> , etc).
?	Corresponde a um caractere do nome de um arquivo ou diretório.	• <code><comando> <nome>?. <extensão></code>	• <code>ls testes/teste?.py</code> : lista <code>teste1.py</code> e <code>teste2.py</code> , mas ignora o <code>teste10.py</code> (pois tem dois caracteres numéricos).
		• <code><comando> <prefixo>? <sufixo></code>	• <code>cp docs/manual_v?.pdf scripts/</code> : copia apenas <code>manual_v1.pdf</code> e <code>manual_v2.pdf</code> .
		• <code><comando> <nome>?. <extensão></code>	• <code>rm scripts/script3.?.py</code> : apaga o <code>script3.8.py</code> , mas ignora o <code>script3.10.py</code> .
[]	Corresponde a qualquer caractere único entre colchetes. Por exemplo, <code>[one]</code> , que significa corresponder a <code>o</code> , <code>n</code> ou <code>e</code> .	• <code><comando> <nome> [letras]. <ext></code>	• <code>ls configs/robo[ACZ].cfg</code> : lista especificamente os arquivos dos robôs A, C e Z (ignorando o <code>roboB.cfg</code>).

Curinga	Função Principal	Estrutura de Uso	Exemplos Práticos (Projeto Sonho/W2G)
[-]	Corresponde a qualquer caractere único dentro do intervalo fornecido entre colchetes. Por exemplo, [0-9]	• <comando> <nome>[A-K]. <ext>	• <code>rm configs/robo[A-C].cfg</code> : apaga as configurações dos robôs A, B e C, mantendo intacto o <code>roboZ.cfg</code> .
	corresponde a qualquer número único de 0 a 9.	• <comando> <nome>[0-9]. <ext>	• <code>cp testes/teste[1-2].py scripts/</code> : copia estritamente os arquivos <code>teste1.py</code> e <code>teste2.py</code> .
[^]	Negação lógica de um caractere único. Por exemplo, [^a-zA-Z] representa a correspondência de um único caractere que não seja uma letra.	• <comando> <nome> [^letras]. <ext>	• <code>ls configs/robo[^A-C].cfg</code> : lista as configurações de qualquer robô, exceto A, B e C (ex: encontra o <code>roboZ.cfg</code>).
		• <comando> <nome>[^0-9]. <ext>	• <code>rm testes/teste[^1].py</code> : apaga arquivos de teste cujo dígito não seja 1 (ex: apaga <code>teste2.py</code>).
{,}	Correspondência não contínua de múltiplos caracteres individuais. Separados por vírgulas.	• <comando> <prefixo> {item1,item2}	• <code>cp docs/{manual_v1,relatorio_erro}.pdf backup/</code> : copia exatamente os dois arquivos listados, sem precisar digitar <code>docs/</code> duas vezes.
{..}	Similar ao intervalo [-], mas focado em gerar/expandir sequências. Por exemplo, {0..9} e {a..z}.	• <comando> <prefixo> {1..3}	• <code>mkdir dir{1..3}</code> : cria os diretórios <code>dir1</code> , <code>dir2</code> e <code>dir3</code> simultaneamente de forma automática.

Um detalhe fundamental no Linux é que **diferentes comandos possuem diferentes níveis de suporte para os estilos de wildcards**. Embora eles sejam processados pelo terminal, a forma como cada ferramenta interage com eles pode variar. Isso acontece principalmente porque alguns comandos servem para manipular itens que já existem, enquanto outros servem para criar novos itens.

Aqui está uma referência rápida de quais comandos suportam quais curingas:

- **ls, cp e mv**: Suportam **todos** os wildcards apresentados (*, ?, [-], [^], { }).
- **find**: Suporta os curingas de busca padrão (*, ?, [-], e [^]).
- **mkdir e touch**: Como são usados para criar arquivos e pastas, seu uso com wildcards é focado na expansão de chaves, como {,} (listas não contínuas) e {..} (intervalos sequenciais), o que é ideal para a criação em massa.

Agora que mapeamos quais ferramentas aceitam quais símbolos, vamos ver como isso funciona na prática criando fluxos de trabalho reais no nosso projeto.

Para visualizar isso na prática e garantir que os comandos funcionem perfeitamente, você precisará preparar o seu ambiente. Como você já aprendeu a navegar e criar elementos no terminal nos capítulos anteriores, este é o seu desafio:

Navegue até a sua pasta **Documentos** e crie a estrutura do nosso projeto **Sonho/W2G** exatamente como apresentada na árvore abaixo, incluindo os subdiretórios e os arquivos vazios:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ tree
.
├── configs
│   ├── roboA.cfg
│   ├── roboB.cfg
│   ├── roboC.cfg
│   └── roboZ.cfg
├── docs
│   ├── manual_v1.pdf
│   ├── manual_v2.pdf
│   └── relatorio_erro.pdf
├── logs
│   ├── erro_critico.log
│   ├── erro_motor.txt
│   ├── sensor_frontal.log
│   └── sensor_traseiro.log
├── scripts
│   ├── main.py
│   ├── script2.7.py
│   ├── script3.10.py
│   └── script3.8.py
└── testes
    ├── teste1.py
    ├── teste10.py
    └── teste2.py
```

2.1 Símbolo: Asterisco (*)

Antes de fazermos alterações, é uma boa prática usar o asterisco com o comando de listagem para ter uma visão geral. Utilizando apenas o *****, o **ls** agrupa o resultado por pastas:

```
(base) thallys@thallys-note:~/Documentos/Sonho$ ls *
configs:
roboA.cfg  roboB.cfg  roboC.cfg  roboZ.cfg

docs:
manual_v1.pdf  manual_v2.pdf  relatorio_erro.pdf

logs:
erro_critico.log  erro_motor.txt  sensor_frontal.log  sensor_traseiro.log
```

```
scripts:
main.py  script2.7.py  script3.10.py  script3.8.py

testes:
teste10.py  teste1.py  teste2.py
```

Também podemos usar o asterisco junto com a barra de diretórios (`*/*`) para ter uma lista rápida de todos os arquivos de subpastas juntamente com seus caminhos relativos:

```
(base) thallys@thallys-note:~/Documentos/Sonho$ ls */*
configs/roboA.cfg  docs/relatorio_erro.pdf  scripts/script2.7.py
configs/roboB.cfg  logs/erro_critico.log   scripts/script3.10.py
configs/roboC.cfg  logs/erro_motor.txt     scripts/script3.8.py
configs/roboZ.cfg  logs/sensor_frontal.log testes/teste10.py
docs/manual_v1.pdf logs/sensor_traseiro.log testes/teste1.py
docs/manual_v2.pdf scripts/main.py          testes/teste2.py
```

Agora, imagine que precisamos fazer um backup rápido de todos os scripts Python do projeto antes de uma grande alteração. Em vez de copiar arquivo por arquivo, usamos o `*` em conjunto com o `mkdir` (para criar o destino) e o `cp` (para copiar em massa tudo que termina em `.py`):

```
# 1. Verificamos o que será afetado na pasta scripts
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls scripts/*.py
scripts/main.py          scripts/script2.7.py          scripts/script3.10.py
scripts/script3.8.py

# 2. Criamos uma pasta de backup na raiz do projeto
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ mkdir backup_scripts

# 3. Copiamos todos os scripts de uma vez para o backup
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cp scripts/*.py
backup_scripts/
```

O asterisco também é excelente para buscas. Se você precisa localizar arquivos relacionados a "erro", mas eles estão espalhados entre `logs` e `docs` e possuem extensões diferentes, você tem duas abordagens. A primeira é usar o `ls` para uma busca mais rápida nas pastas imediatas:

```
# O primeiro asterisco pesquisa dentro de qualquer pasta, os outros buscam
a palavra no nome
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls */*erro*
docs/relatorio_erro.pdf  logs/erro_critico.log  logs/erro_motor.txt
```

Porém, se você não faz ideia de quão profunda a pasta está escondida na árvore do projeto, utilizando o comando `find` apresentado na última aula. Ele vasculha todo o diretório atual (`.`) procurando pelo padrão

exato de nome:

```
# O find com o asterisco pesquisa a palavra em qualquer parte do nome do
arquivo
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ find . -name "*erro*"
./docs/relatorio_erro.pdf
./logs/erro_critico.log
./logs/erro_motor.txt
```

2.2 Símbolo: Interrogação (?)

A interrogação substitui **estritamente um único caractere**. Suponha que os testes iniciais (teste 1 a 9) já foram validados e você precisa movê-los com o comando `mv` para a pasta principal de scripts, mas quer deixar os testes mais recentes (como o teste 10 em diante) na pasta atual.

```
# 1. Verificamos quem tem exatamente UM caractere numérico antes do ponto
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls testes/teste?.py
testes/teste1.py testes/teste2.py

# 2. Movemos apenas os testes validados para a pasta de scripts
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ mv testes/teste?.py
scripts/
```

- **Obs.:** Note que o `teste10.py` ficou de fora, pois exigiria `??` (dois caracteres) para ser identificado.

2.3 Símbolo: Colchetes de Conjunto ([]), Intervalo ([-]) e Negação ([^])

Os colchetes são ideais quando precisamos filtrar arquivos baseados em um conjunto específico de caracteres ou em uma sequência. Diferente das chaves que servem para gerar/expandir comandos, os colchetes são usados estritamente para buscar o que já existe, avaliando apenas uma única posição (um caractere) no nome do arquivo. Digamos que você precisa trabalhar com as configurações dos robôs:

2.3.1 A lógica base do Conjunto ([]):

Para listar estritamente os robôs A, C e Z (um conjunto específico que pula o B), usamos letras soltas dentro dos colchetes:

```
# O terminal busca arquivos que tenham exatamente A, C ou Z naquela posição
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls
configs/robo[ACZ].cfg
configs/roboA.cfg configs/roboC.cfg configs/roboZ.cfg
```

2.3.2 Usando o circunflexo [^] para Negação:

Se quiséssemos fazer exatamente o oposto, listar as configurações que **não** fossem da primeira geração (robôs A, B ou C), usaríamos o símbolo de negação `^` logo no início dos colchetes, combinado com o intervalo `A-C`:

```
# O circunflexo inverte a busca: encontra qualquer robô, exceto A, B e C
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/robo[^A-C].cfg
configs/roboZ.cfg
```

2.3.3 Usando o traço [-] para Intervalos:

Quando temos uma sequência natural, não precisamos digitar tudo. Caso quiséssemos apagar definitivamente as configurações obsoletas de toda a primeira geração (de A até C), é uma boa prática primeiro listar os arquivos para confirmar e, em seguida, aplicar o intervalo junto com o comando `rm`:

```
# 1. Primeiro, listamos para confirmar os arquivos da sequência de A até C
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/robo[A-C].cfg
configs/roboA.cfg configs/roboB.cfg configs/roboC.cfg
```

```
# Removemos cirurgicamente os arquivos na sequência de A até C (A, B e C)
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ rm configs/robo[A-C].cfg
```

Um alerta importante sobre a literalidade do terminal:

Os colchetes buscam apenas por **um** único caractere dentro daquele intervalo. Observe o que acontece se tentarmos listar nossos scripts Python buscando pela versão maior que 3.0 usando o intervalo numérico de 0 a 9:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls scripts/script3.[0-9].py
scripts/script3.8.py
```

Perceba que o `script3.10.py` foi ignorado. Isso acontece porque o número `10` possui dois caracteres, e a regra `[0-9]` busca por apenas um. O terminal faz exatamente o que você pede, de forma literal, sem assumir as suas intenções!

2.4 Símbolo: Expansão de Chaves (`{ }`, `{,}` e `{..}`)

Ao invés de buscar arquivos que possuem nomes parecidos, as chaves são as melhores amigas dos comandos de criação (`mkdir`, `touch`) e de cópia múltipla (`cp`), pois nos permitem gerar listas, expandir sequências ou agrupar palavras que não seguem um padrão único.

2.4.1 A lógica base das Chaves ({ }):

As chaves funcionam como um **multiplicador** no terminal. O que está do lado de fora das chaves (como o caminho de uma pasta ou a extensão de um arquivo) é mantido, enquanto o que está do lado de dentro é expandido e distribuído, economizando muita digitação.

Para visualizar como o terminal faz essa montagem na prática, podemos usar o comando `ls`. O terminal primeiro expande as chaves e, em seguida, lista exatamente os arquivos alvo:

```
# O terminal repete o prefixo "configs/robo" e o sufixo ".cfg" para cada
letra dentro das chaves
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$      ls
configs/robo{A,B,C}.cfg
configs/roboA.cfg  configs/roboB.cfg  configs/roboC.cfg
```

2.4.2 Usando a vírgula {, } para listas:

Se precisarmos criar várias pastas de organização ao mesmo tempo e, em seguida, copiar os manuais em PDF e um relatório de erro para a pasta de entrega (**delivery**), separamos as opções por vírgula:

```
# O mkdir com chaves cria múltiplas pastas diferentes de uma só vez
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$      mkdir
{delivery,revisao,arquivados}

# No cp, as chaves evitam que você tenha que digitar "docs/" repetidas
vezes
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$      cp
docs/{manual_v1,relatorio_erro}.pdf delivery/
```

2.4.3 Usando os dois pontos {..} para sequências:

Por fim, a expansão de sequência é incrivelmente poderosa para a criação em massa. Vamos usar o `mkdir` para gerar pastas para as próximas cinco fases de testes do robô e o `touch` para criar três novos arquivos de log para sensores extras instantaneamente:

```
# Criando 5 pastas sequenciais com um único comando
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$      mkdir
testes/fase_{1..5}

# Criando 3 arquivos de log sequenciais instantaneamente
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$      touch
logs/sensor_extra_{1..3}.log
```

2.5 O Resultado Final

Após executar todos esses fluxos de trabalho combinando comandos (`ls`, `cp`, `mkdir`, `mv`, `rm`, `find`, `touch`) com os diferentes símbolos (Wildcards), o ambiente do nosso projeto foi totalmente transformado. Se você rodar o comando `tree` agora, este deve ser o estado atual da sua pasta:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ tree
```

```
.
├── arquivados
├── backup_scripts
│   ├── main.py
│   ├── script2.7.py
│   ├── script3.10.py
│   └── script3.8.py
├── configs
│   └── roboZ.cfg
├── delivery
│   ├── manual_v1.pdf
│   └── relatorio_erro.pdf
├── docs
│   ├── manual_v1.pdf
│   ├── manual_v2.pdf
│   └── relatorio_erro.pdf
├── logs
│   ├── erro_critico.log
│   ├── erro_motor.txt
│   ├── sensor_extra_1.log
│   ├── sensor_extra_2.log
│   ├── sensor_extra_3.log
│   ├── sensor_frontal.log
│   └── sensor_traseiro.log
├── revisao
├── scripts
│   ├── main.py
│   ├── script2.7.py
│   ├── script3.10.py
│   ├── script3.8.py
│   ├── teste1.py
│   └── teste2.py
└── testes
    ├── fase_1
    ├── fase_2
    ├── fase_3
    ├── fase_4
    ├── fase_5
    └── teste10.py
```

```
14 directories, 24 files
```

3. Links: Atalhos e Espelhos no Sistema

Trabalhando com robótica, é muito comum que scripts principais dependam de arquivos de configuração, manuais ou logs que estão "escondidos" profundamente em subdiretórios. Navegar toda hora por esses

caminhos longos é cansativo, e duplicar (copiar) os arquivos não é uma opção inteligente, pois desperdiça espaço em disco e gera versões desatualizadas.

Para resolver isso, utilizamos os **Links** no Linux. Eles permitem que um arquivo com conteúdo dinâmico (ou escondido) seja acessado facilmente a partir de qualquer outro lugar usando um nome estático e acessível. Vale ressaltar um ponto fundamental antes de tudo: **um link não é uma cópia isolada**. Qualquer modificação feita em um arquivo ou diretório **através do link** será aplicada diretamente no original. Editar, renomear ou alterar conteúdo pelo atalho é exatamente o mesmo que fazer isso na fonte. O link é apenas uma porta de entrada diferente para o mesmo dado real.

Existem dois tipos fundamentais de links no ecossistema Linux/UNIX:

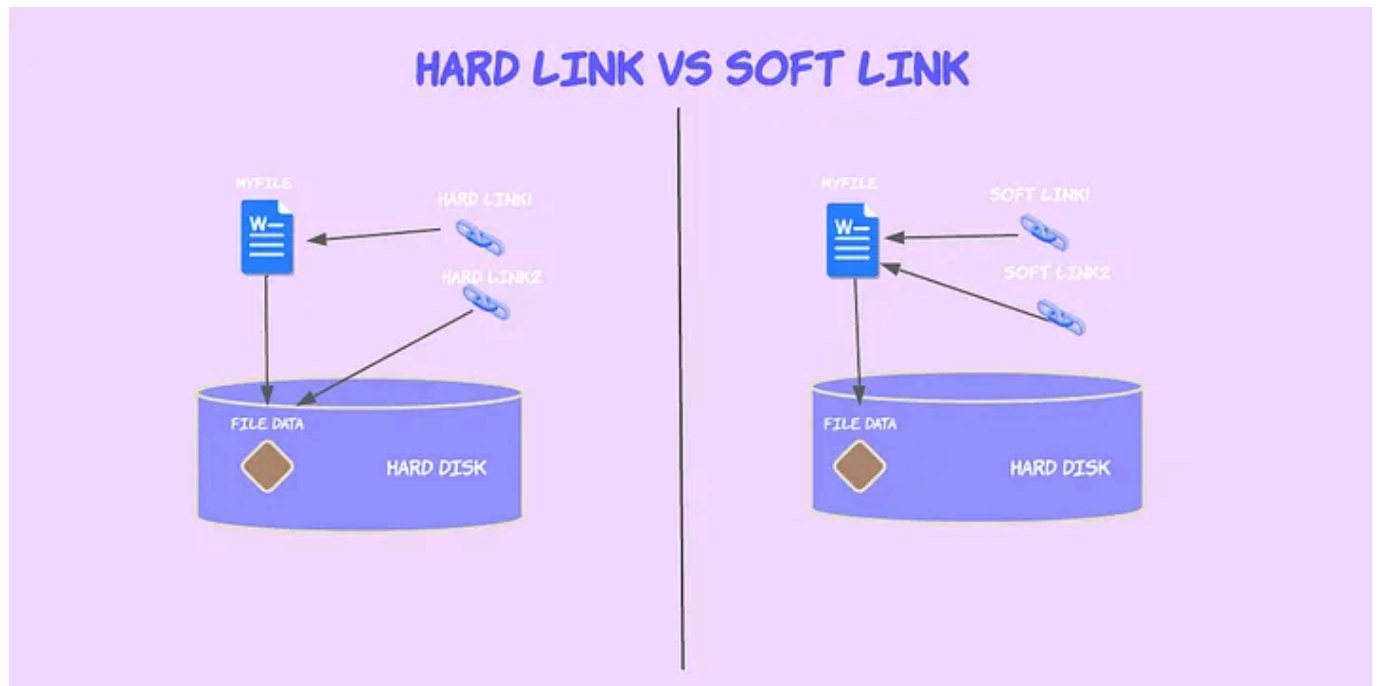


Figura 1: Hard Links x Soft Links.

- **Hard Links (Links Rígidos):** Neste modelo, existe apenas um arquivo físico real no disco rígido, mas ele ganha duas ou mais "portas de entrada" (nomes oficiais). Se você excluir o arquivo original, o outro hard link continuará existindo e acessando os dados perfeitamente. Contudo, eles têm limitações severas: não podem ser criados para diretórios (pastas) e não funcionam entre discos ou partições diferentes.
- **Soft Links ou Symbolic Links (Links Simbólicos):** Este é o modelo que você mais utilizará no dia a dia. Ele funciona exatamente como o **Atalho do Windows**. Um **symlink** é apenas um arquivo minúsculo que guarda o caminho apontando para o arquivo ou pasta original. Ele pode apontar para diretórios inteiros e cruzar discos diferentes sem problemas.

A grande diferença prática é, se você **deletar o atalho (symlink)**, o arquivo original continua intacto. No entanto, se você **deletar o arquivo original**, o symlink ficará **quebrado**, apontando para o nada.

3.1 Criando Links Simbólicos para Arquivos

Para criar um link simbólico, utilizamos o comando **ln** (link) acompanhado da flag **-s** (simbólico). A sintaxe padrão segue a lógica de **Origem** (o arquivo real) e depois **Destino** (o nome do atalho):

```
ln -s <caminho_do_arquivo_original> <nome_do_atalho_a_ser_criado>
```

Para exemplificar, imagine um cenário comum: você acabou de criar uma nova pasta para a versão 2.0 do robô, chamada `W2G_v2`, paralela à pasta original `W2G`. No entanto, você quer continuar utilizando o script principal da versão anterior sem precisar copiá-lo (o que geraria versões duplicadas e dessincronizadas no futuro).

Vamos entrar na nova pasta e criar um atalho que puxa o script da pasta antiga:

Se você não especificar um nome de destino, o comando `ln` criará automaticamente um atalho no seu diretório atual com o mesmo nome do arquivo original.

```
# Criamos o atalho apontando para o arquivo lá no projeto antigo (W2G)
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ ln -s
../W2G/scripts/main.py rodar_robo_v1.py
```

Como podemos ter certeza de que o atalho foi criado corretamente e não é apenas um arquivo de texto comum? É aqui que entra o comando de listagem. Conforme a própria documentação de ajuda do comando `ls` (`ls --help`) nos ensina, a opção `-l` ativa o "formato de lista longa", trazendo metadados fundamentais:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ ls -l
rodar_robo_v1.py
lrwxrwxrwx 1 thallys thallys 23 mar  3 09:40 rodar_robo_v1.py ->
../W2G/scripts/main.py
```

Perceba duas coisas vitais nesta saída detalhada:

1. O primeiríssimo caractere da linha é um `l` minúsculo (de link), indicando ao sistema que este não é um arquivo físico, mas sim um ponteiro.
2. No final da linha, há uma seta explícita `->` mostrando visualmente para qual caminho o atalho está redirecionando o sistema.

A partir de agora, qualquer alteração que você fizer executando ou editando o `rodar_robo_v1.py` dentro da versão 2.0 será, na verdade, aplicada ao arquivo original lá na pasta `W2G/scripts/`.

Vale notar o número `1` que aparece logo após as permissões nas duas saídas. Ele representa a **contagem de hard links** daquele item no disco. Soft Links **não** incrementam esse contador — eles são arquivos independentes identificados apenas pelo `1` no início da string de permissões e pela seta `->` no final da linha. Já para arquivos comuns, esse número indica quantos Hard Links apontam para o mesmo dado físico no disco. Um arquivo recém-criado começa com `1`. Se um segundo hard link fosse criado para o `main.py`, esse contador subiria para `2`, indicando que o dado físico tem duas "portas de entrada" no sistema. O dado só é realmente removido do disco quando esse contador chega a zero.

Para visualizar, compare as duas saídas abaixo. O `main.py` foi criado normalmente e tem contador `1`. O `main_backup.py` é um hard link criado para o mesmo dado, e ambos passam a exibir `2`:

```
-rwxrwxr-- 2 thallys thallys 35 mar 7 14:22 main.py
T
|
└─ Contador subiu para 2: dois nomes apontam para o mesmo dado
no disco
└─ Arquivo comum (-)

lrwxrwxrwx 1 thallys thallys 23 mar 7 14:25 rodar_robo_v1.py ->
../W2G/scripts/main.py
T
|
└─ Seta
indica o destino do symlink
└─ Contador permanece 1: soft link não incrementa a contagem
└─ Link simbólico (l)
```

3.2 Criando Links para Diretórios (Pastas)

Os symlinks não se limitam a arquivos, eles são extremamente úteis para **transportar** pastas inteiras. Suponha que o seu novo robô 2.0 ainda precise ler todas as configurações de hardware dos robôs da primeira geração. Em vez de copiar tudo, podemos espelhar a pasta `configs` inteira para o nosso novo ambiente de trabalho:

```
# Criando um atalho chamado "configs_legacy" que aponta para a pasta
original
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ ln -s ../W2G/configs
configs_legacy
```

Se você listar o conteúdo ou entrar na pasta `configs_legacy` agora, verá exatamente os mesmos arquivos (`roboA.cfg`, `roboZ.cfg`) que estão no projeto original. Uma mudança lá, reflete imediatamente aqui!

3.3 Atualizando e Sobrecrevendo um Link

Em ambientes de desenvolvimento, as dependências mudam rápido. Digamos que você percebeu que o script principal `main.py` antigo não serve mais, e agora você quer que o seu atalho `rodar_robo_v1.py` passe a apontar para uma versão específica do Python, o `script3.10.py`.

Se você tentar usar o comando `ln -s` novamente com o mesmo nome de destino, o terminal dará um erro dizendo que o arquivo já existe.

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ ln -s
../W2G/scripts/script3.10.py rodar_robo_v1.py
ln: falha ao criar link simbólico 'rodar_robo_v1.py': Arquivo existe
```

Para forçar a sobrescrita do link sem precisar apagá-lo primeiro, adicionamos a flag `-f` (force):

```
# Atualiza o link existente para apontar para o novo alvo
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ ln -f -s
../W2G/scripts/script3.10.py rodar_robo_v1.py
```

3.4 Removendo Links e Lidando com "Links Quebrados"

Como vimos, o symlink é totalmente dependente do arquivo original. Se o arquivo original mudar de nome, de lugar ou for deletado, o atalho perde sua âncora e vira um **link quebrado**.

Vamos simular isso apagando o arquivo original no projeto antigo para o qual o nosso link aponta atualmente:

```
# Apagando o arquivo original lá no projeto raiz
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ rm
../W2G/scripts/script3.10.py
```

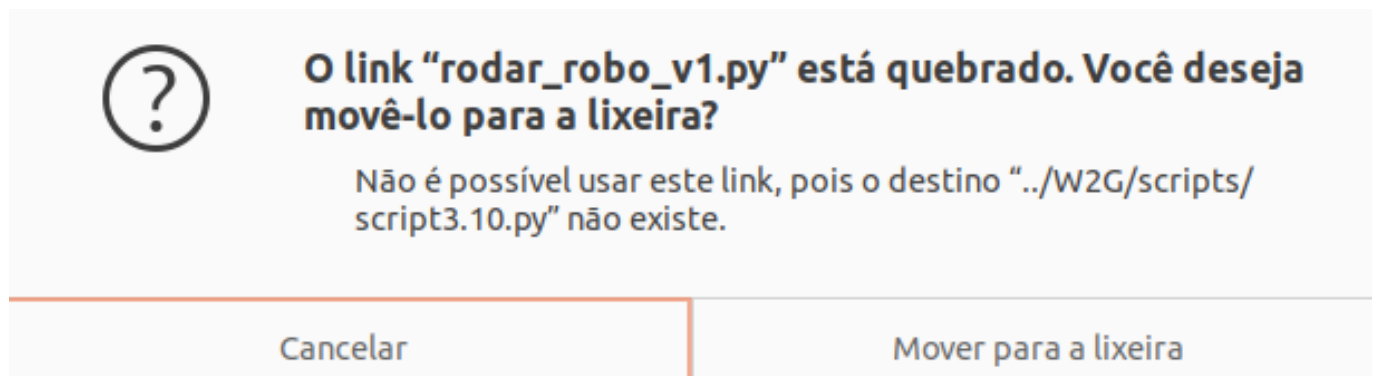


Figura 2: Link Quebrado.

Se você tentar ler o `rodar_robo_v1.py` agora, receberá um erro clássico: **No such file or directory (O arquivo ou diretório não existe)**. O atalho está lá, mas o caminho que ele guarda leva a lugar nenhum.

Em sistemas com centenas de arquivos cruzados, encontrar esses links fantasmas manualmente é impossível. É aqui que o comando `find` entra de forma brilhante com o parâmetro `-xtype l` (que pesquisa especificamente por links que apontam para o vazio):

```
# Varre o diretório atual em busca de links quebrados
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ find . -xtype l
./rodar_robo_v1.py
```

Uma vez identificado o "fantasma", como nos livramos dele de forma segura? Existem duas maneiras de remover um link:

- **Usando o unlink (Específico para links):**

É o método mais tradicional e seguro quando se lida com um único link. Funciona tanto para links de arquivos quanto para links de diretórios.

Para remover um link de arquivo:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ unlink  
rodar_robo_v1.py
```

Para remover um link de diretório:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ unlink  
configs_legacy
```

- **Usando o rm (Comando padrão de remoção):**

O grande benefício do **rm** é poder apagar múltiplos links de uma só vez usando wildcards.

Lembra do **configs_legacy** que criamos apontando para a pasta original? Para apagá-lo, basta usar o nome do link diretamente:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G_v2$ rm configs_legacy
```

Dominar os links simbólicos, compreendendo como os arquivos se interconectam e como visualizar isso através do **ls -l**, é um dos grandes saltos na sua jornada. Eles são uma das melhores ferramenta quando trabalhamos com código, mantendo configurações centralizadas e evitando a temida duplicação de dados!

4. Grupos e Permissões de Arquivos: Quem pode fazer o quê?

O Linux é um sistema operacional multiusuário, logo inclui mecanismos que protegem dados contra acesso não-autorizado. Cada diretório e arquivo têm um usuário associado, chamado de proprietário (**owner**), que criou o diretório ou arquivo. Cada usuário pertence a um ou mais conjuntos de usuários, conhecidos como **grupos**. Cada diretório e arquivo tem um grupo associado, que é atribuído quando o diretório ou arquivo é criado.

Para ver qual é o seu usuário atual e a quais grupos ele pertence, use:

```
# Exibe o usuário atual  
(base) thallys@thallys-note:~$ whoami  
thallys  
  
# Exibe o usuário e todos os grupos aos quais ele pertence  
(base) thallys@thallys-note:~$ id  
uid=1000(thallys)                                gid=1000(thallys)  
grupos=1000(thallys),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),122(lpadmin)
```

```
min), 135(lxd), 136(sambashare), 999(docker)
```

A saída do `id` traz três informações fundamentais sobre a identidade do usuário no sistema:

Campo	Definição
uid (User ID)	Identificador numérico único do usuário. O sistema usa esse número internamente, visto que o nome é apenas um apelido legível para os usuários. IDs abaixo de 1000 são reservados para o sistema e os usuários comuns começam a partir do 1000.
gid (Group ID)	Identificador numérico do grupo padrão do usuário. Todo arquivo ou diretório criado por ele recebe esse grupo automaticamente.
grupos	Lista todos os grupos aos quais o usuário pertence, além do grupo padrão. Cada grupo tem seu próprio ID e concede um conjunto diferente de privilégios.

Por exemplo, o `uid=1000(thallys)` indica que é um usuário comum criado na instalação. O `gid=1000(thallys)` mostra o grupo padrão, que por convenção recebe o mesmo nome do usuário. Já o campo `grupos=1000(thallys),4(adm),27(sudo),999(docker)` revela que o usuário pertence a quatro grupos simultaneamente: o grupo pessoal `thallys`, o grupo `adm` que concede acesso a logs do sistema, o grupo `sudo` que permite executar comandos como administrador e o grupo `docker` que permite gerenciar contêineres sem precisar de `sudo` (você verão isso no futuro). Essa estrutura de grupos é o que permite ao Linux controlar com precisão o que cada pessoa pode ou não fazer em cada arquivo do sistema.

Cada grupo concede um tipo diferente de privilégio no sistema. Se quiser ver todos os grupos existentes na máquina, não apenas os seus, o arquivo `/etc/group` guarda essa lista completa, onde cada linha segue o formato `nome_do_grupo : senha : ID : membros`. Para filtrar apenas os grupos aos quais você pertence, combine o `cat` com o `grep` passando o seu nome de usuário como argumento, `thallys` no nosso caso:

```
(base) thallys@thallys-note:~$ cat /etc/group | grep thallys
adm:x:4:syslog,thallys
cdrom:x:24:thallys
sudo:x:27:thallys
docker:x:999:thallys
thallys:x:1000:
```

Olhando para os IDs numéricos dessa saída, perceba que os grupos do sistema como `adm` e `sudo` têm IDs baixos (4 e 27), enquanto o grupo pessoal `thallys` tem ID 1000. Isso não é coincidência, pois o Linux segue uma convenção rígida para organizar esses números:

Faixa de ID	Tipo	Exemplos
0	Superusuário	<code>root</code>
1 – 999	Usuários e grupos do sistema	<code>adm</code> , <code>sudo</code> , <code>docker</code> , entre outros

Faixa de ID	Tipo	Exemplos
1000+	Usuários comuns	Como thallys e outros usuários criados na instalação

Essa tabela não é apenas uma curiosidade técnica. Ela revela a hierarquia de poder do sistema: quanto menor o **ID**, mais profundo e crítico é o papel daquele usuário ou grupo. E no topo absoluto dessa hierarquia, com **ID zero**, está o **root**.

Superusuário e a hierarquia de privilégios

O **root** é o **superusuário** do Linux, ele é o único com acesso irrestrito a absolutamente tudo no sistema. Logo, ele pode instalar programas, modificar arquivos do sistema, criar e deletar usuários, alterar permissões de qualquer arquivo. Ele tem **uid=0**, e é justamente esse número que o kernel usa para identificá-lo e liberar todos os acessos, independente de qualquer permissão configurada.

Por ser tão poderoso, usar o **root** diretamente no dia a dia é considerado uma má prática, pois um comando errado pode inutilizar o sistema inteiro sem nenhuma confirmação. Por isso, o Linux adota uma hierarquia de acesso com três níveis principais:

Nível	Quem é	Como age no sistema
root	Superusuário absoluto (uid=0)	Acesso total e irrestrito a tudo, sem nenhuma barreira
sudoers	Usuários comuns no grupo sudo	Executam comandos pontuais como root usando o prefixo sudo
usuários comuns	Todos os demais	Acesso restrito apenas aos seus próprios arquivos e ao que lhes foi explicitamente permitido

É importante entender que um **sudoer** não é **root**. Ele continua sendo um usuário comum, mas com uma permissão especial: a de pedir ao sistema que execute um comando específico com privilégios de **root**. Essa distinção existe por segurança, pois se o seu **usuário** fosse **root** diretamente, qualquer programa mal-intencionado ou comando digitado errado teria poder total sobre o sistema sem nenhuma barreira. Ao usar o **sudo**, você adiciona uma camada de intenção consciente: o sistema exige sua senha, registra o que foi feito nos logs e limita o privilégio àquele único comando. É a diferença entre ter uma chave mestra guardada no bolso e precisar buscá-la toda vez que quiser usá-la.

Na prática, você interage com essa hierarquia de duas formas:

- **sudo <comando>**: eleva a permissão apenas para aquele comando específico. O sistema pede sua senha para confirmar a intenção e registra a ação nos logs. É o uso recomendado para tarefas administrativas pontuais.

```
# Instala um pacote com privilégio de root por um único comando
(base) thallys@thallys-note:~$ sudo apt install htop
```

O **htop** é usado aqui apenas como exemplo de instalação. Trata-se de um monitor de processos interativo e visual para o terminal, que exibe em tempo real o uso de CPU, memória

e todos os processos rodando no sistema, sendo muito mais agradável de usar do que o `top` nativo. Caso queira explorá-lo por conta própria, há um bom material em <https://blog.ironlinux.com.br/o-comando-htop-no-linux/> ou você pode consultar o manual diretamente pelo terminal com `man htop`. Vale lembrar que instalar qualquer pacote com `sudo` exige atenção: sempre verifique a procedência do que está instalando, pois um pacote malicioso rodando com privilégios elevados pode comprometer o sistema inteiro.

- **sudo su**: abre uma sessão completa como `root`, transformando o terminal atual em um terminal de superusuário até você digitar `exit`. É útil quando você precisa executar muitos comandos administrativos seguidos, mas exige atenção redobrada — qualquer comando digitado ali tem poder total.

```
# Entra na sessão root
(base) thallys@thallys-note:~$ sudo su
root@thallys-note:/home/thallys#

... comandos ...

# Sai da sessão root e volta ao usuário comum
root@thallys-note:/home/thallys# exit
(base) thallys@thallys-note:~$
```

Essa estrutura de **usuários e grupos** é o que permite ao Linux controlar com precisão quem pode fazer o quê em cada arquivo do sistema. E é exatamente aí que entram as **permissões de acesso**.

Vale destacar um cenário prático que você encontrará com frequência: arquivos e diretórios criados com `sudo` pertencem ao `root`, não ao seu usuário. Isso significa que, mesmo sendo o criador do arquivo, você não conseguirá lê-lo, editá-lo ou executá-lo sem elevar novamente o nível de permissão. Nesses casos, você precisará do `sudo` não só para criar, mas também para qualquer operação posterior sobre esses arquivos. Veremos exemplos concretos disso nas seções abaixo.

As **permissões de acesso** determinam exatamente quais operações cada tipo de usuário pode realizar em um diretório ou arquivo. O sistema é restrito, ou seja, somente as permissões listadas podem ser concedidas e qualquer outra operação é automaticamente proibida. Um detalhe importante é que as permissões trabalham de maneira diferente para diretórios e arquivos. A permissão `r`, por exemplo, indica a capacidade de ler o conteúdo de um arquivo, mas em um diretório ela indica a capacidade de listar o que há dentro dele. Um diretório ou arquivo pode ter mais de uma permissão combinada: um usuário com permissão `rw`, por exemplo, pode ler e gravar um arquivo, mas não executá-lo, pois o `x` de execução não está presente.

4.1 Lendo a String de Permissões

O melhor jeito de entender permissões é observando-as na prática. Vamos verificar como estão as permissões do `main.py` no nosso projeto usando o `ls -l`, onde o `-l` ativa o formato de lista longa, exibindo os metadados completos de cada arquivo:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l scripts/main.py
-rw-rw-r-- 1 thallys thallys 0 Mar  3 10:00 scripts/main.py
```

A primeira coluna, `-rw-rw-r--`, é a **string de permissões** — a identidade de segurança daquele arquivo. Ela tem sempre 10 caracteres e é lida da seguinte forma:

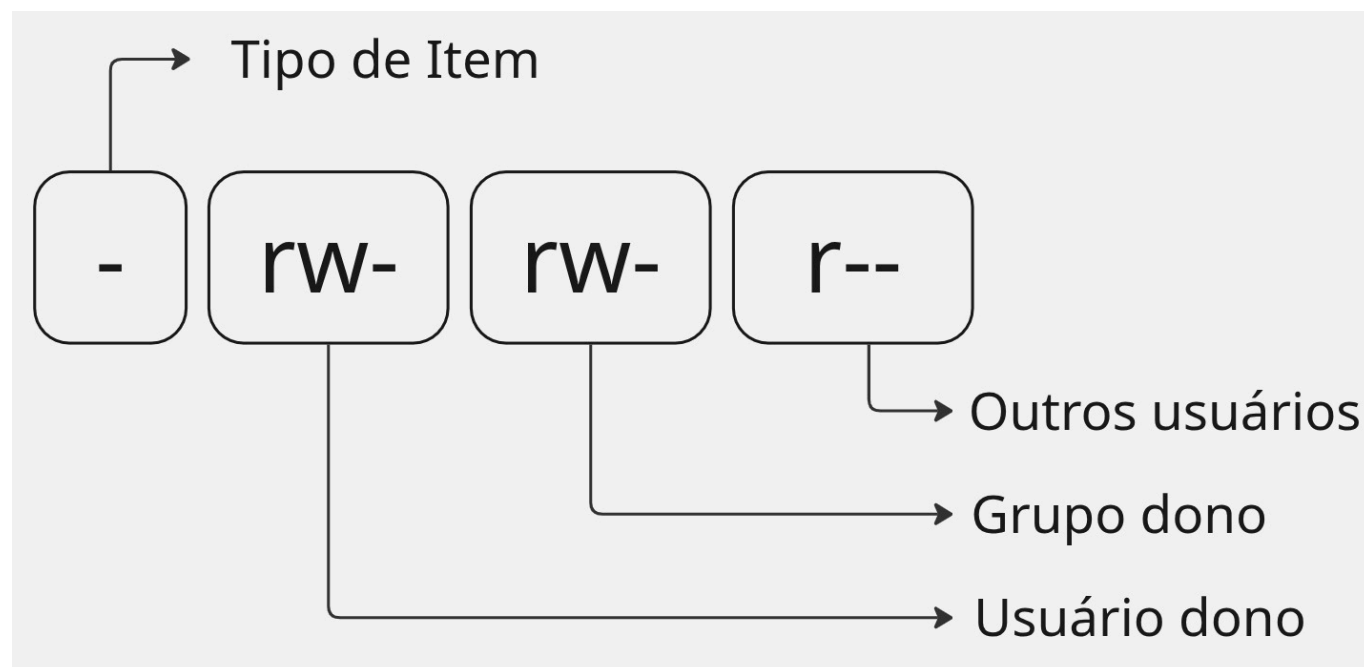


Figura 3: Estrutura da string de permissões no Linux.

A estrutura dessa string é lida da seguinte forma:

Posição	Conteúdo	Significado
1º caractere	<code>- / d / l</code>	Tipo do item: arquivo comum, diretório ou link simbólico
2º ao 4º caractere	<code>rwX</code>	Permissões do Usuário (u) — o dono do arquivo
5º ao 7º caractere	<code>rwX</code>	Permissões do Grupo (g) — usuários do grupo associado
8º ao 10º caractere	<code>rwX</code>	Permissões dos Outros (o) — qualquer outro usuário do sistema

Voltando ao nosso exemplo `-rw-rw-r--`, o sistema está nos dizendo:

- `-` → é um arquivo comum.
- `rw-` (Usuário): o dono `thallys` pode ler e escrever, mas **não executar**.
- `rw-` (Grupo): o grupo `thallys` também pode ler e escrever.
- `r--` (Outros): qualquer outra pessoa só pode ler.

Dentro de cada bloco de três posições, as letras que aparecem são as **flags de permissão**. Um `-` em qualquer posição significa que aquela operação foi negada:

Letra	Nome	O que faz em um Arquivo	O que faz em um Diretório
<code>r</code>	Read (Leitura)	Permite ler o conteúdo (ex: <code>cat</code>).	Permite listar o conteúdo (ex: <code>ls</code>).

Letra	Nome	O que faz em um Arquivo	O que faz em um Diretório
w	Write (Escrita)	Permite modificar, gravar ou apagar o conteúdo.	Permite criar, renomear ou deletar arquivos dentro dele.
x	eXecute (Execução)	Permite rodar o arquivo como um programa/script.	Permite entrar na pasta (ex: cd).
-	Negado	A permissão foi negada para este grupo.	A permissão foi negada para este grupo.

4.2 Alterando Permissões com o **chmod**

Como o **main.py** é o script principal do nosso projeto, vamos tentar rodá-lo diretamente. Sem a permissão de execução, o Linux nos barra imediatamente:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/main.py
bash: ./scripts/script3.10.py: Permissão negada
```

Para liberar a execução, usamos o **chmod** (**Change mode**). Ele pode ser usado de duas formas: pelo **Modo Simbólico** (letras) ou pelo **Modo Octal** (números).

Modo Simbólico

A lógica do modo simbólico segue a equação: **Quem + Ação + Permissão**, onde:

- **Quem:** **u** (user/dono), **g** (group), **o** (others) ou **a** (all — todos).
- **Ação:** **+** (adiciona), **-** (remove), **=** (define exatamente essas, removendo as demais).
- **Permissão:** **r**, **w** ou **x**.

Por enquanto, vamos adicionar a permissão de execução para o usuário com o modo simbólico:

```
chmod u+x scripts/main.py
```

Porém, apenas a permissão **x** não é suficiente. Ao executar **./scripts/main.py** diretamente, o sistema precisa saber **qual interpretador** usar para ler o arquivo. Sem essa informação, ele tenta rodá-lo como um script de shell e falha com um erro de sintaxe:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/main.py
./scripts/main.py: 2: Syntax error: word unexpected (expecting ")")
```

Ao executar **./scripts/main.py** diretamente, o sistema precisa saber **qual interpretador** usar para ler o arquivo. Sem essa informação, ele tenta rodá-lo como um script de shell e falha com um erro de sintaxe. A solução é adicionar o **shebang** (**#!**) na primeira linha do arquivo — uma instrução que aponta o caminho do interpretador correto:

```
#!/usr/bin/env python3
print("Sonho, bem-vindo")
```

Com o shebang no lugar e a permissão `x` concedida, o script finalmente roda diretamente:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/main.py
Sonho, bem-vindo
```

Modo Octal

Embora o modo simbólico seja intuitivo, existe uma forma ainda mais direta de definir permissões: o **Modo Octal**. Em vez de encadear letras e operadores, você configura os três grupos — Usuário, Grupo e Outros — de uma só vez com apenas três dígitos. É o formato mais utilizado no dia a dia.

A lógica é simples, cada permissão tem um valor fixo, e o número do bloco é a soma dos valores ativos:

Símbolo	Permissão	Valor
<code>r</code>	Leitura	4
<code>w</code>	Escrita	2
<code>x</code>	Execução	1
<code>-</code>	Nenhuma	0

Por exemplo, se você quer dar leitura e escrita (`rw-`), soma $4 + 2 + 0 = 6$. Se quer permissão total (`rwX`), soma $4 + 2 + 1 = 7$. A tabela abaixo mostra todas as combinações possíveis:

Combinação	Cálculo	Valor Octal
<code>rwX</code>	$4 + 2 + 1$	7
<code>rw-</code>	$4 + 2 + 0$	6
<code>r-X</code>	$4 + 0 + 1$	5
<code>r--</code>	$4 + 0 + 0$	4

Com esses valores em mente, a tabela completa de todas as permissões possíveis fica assim:

Valor Octal	Valor Binário (rwX)	Caracteres	Significado
0	000	<code>---</code>	Nenhuma permissão de acesso
1	001	<code>--X</code>	Permissão de execução
2	010	<code>-W-</code>	Permissão de gravação
3	011	<code>-WX</code>	Permissão de gravação e execução

Valor Octal	Valor Binário (rwx)	Caracteres	Significado
4	100	r--	Permissão de leitura
5	101	r-x	Permissão de leitura e execução
6	110	rw-	Permissão de leitura e gravação
7	111	rwX	Permissão de leitura, gravação e execução

Na prática, o `chmod` no modo octal recebe três dígitos seguido, um para cada grupo. Veja como isso simplifica os comandos:

Exemplo 1: Configuração padrão para scripts executáveis (755)

Queremos que o dono do `script2.7.py` tenha permissão total (7 = `rwX`), mas que o grupo e os outros possam apenas ler e executar, sem modificar o código (5 = `r-x`). Em vez de três comandos separados, fazemos tudo de uma vez:

Antes:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l
scripts/script2.7.py
-rw-rw-r-- 1 thallys thallys 0 Mar  3 10:00 scripts/script2.7.py
```

Aplicamos o `chmod 755`:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ chmod 755
scripts/script2.7.py
```

Depois:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l
scripts/script2.7.py
-rwxr-xr-x 1 thallys thallys 0 Mar  3 10:00 scripts/script2.7.py
```

Exemplo 2: Configuração de segurança máxima para configurações (600)

Imagine que o arquivo `configs/roboZ.cfg` tem senhas da rede do robô. Queremos que apenas o dono possa ler e escrever (6 = `rw-`), e que ninguém mais tenha qualquer acesso (0 = `---`):

Antes

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l configs/roboZ.cfg
-rw-rw-r-- 1 thallys thallys 0 Mar  3 10:00 configs/roboZ.cfg
```

Aplicamos o `chmod 600`

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ chmod 600  
configs/roboZ.cfg
```

Depois

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l configs/roboZ.cfg  
-rw----- 1 thallys thallys 0 Mar  3 10:00 configs/roboZ.cfg
```

Exemplo 3: Aplicando o parâmetro recursivo (-R)

Imagine ter que dar permissão de execução para todos os scripts da pasta `backup_scripts/` de uma só vez. Fazer isso um por um seria inviável. Assim como em outros comandos do Linux, o `chmod` possui o parâmetro recursivo `-R` (maiúsculo). Ele aplica a mesma permissão à pasta especificada e a **todos** os arquivos e subpastas que estiverem dentro dela de uma só vez.

Antes:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l backup_scripts/  
total 16  
-rw-rw-r-- 1 thallys thallys 0 mar  7 14:20 main.py  
-rw-rw-r-- 1 thallys thallys 0 mar  7 14:20 script2.7.py  
-rw-rw-r-- 1 thallys thallys 0 mar  7 14:20 script3.10.py  
-rw-rw-r-- 1 thallys thallys 0 mar  7 14:20 script3.8.py
```

Aplicamos o `chmod -R 744` para dar permissão de execução ao dono em todos os arquivos de uma vez:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ chmod -R 744  
backup_scripts/
```

Depois, todos os arquivos foram alterados de uma só vez:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l backup_scripts/  
total 16  
-rwxr--r-- 1 thallys thallys 0 mar  7 14:20 main.py  
-rwxr--r-- 1 thallys thallys 0 mar  7 14:20 script2.7.py  
-rwxr--r-- 1 thallys thallys 0 mar  7 14:20 script3.10.py  
-rwxr--r-- 1 thallys thallys 0 mar  7 14:20 script3.8.py
```

4.3 Mudando a Propriedade: `chown` e `chgrp`

Até agora vimos como controlar o que cada usuário pode fazer com um arquivo. Mas há situações em que você precisa ir além e transferir a própria posse do arquivo ou diretório para outro usuário ou grupo. Para isso, o Linux oferece dois comandos complementares: o **chown** (**Change owner**), que muda o usuário dono, e o **chgrp** (**Change group**), que muda apenas o grupo associado.

Exemplo 1: Retirando a exigência de sudo de uma pasta

Imagine que, por distração, você usou o **sudo** na hora de criar a pasta **testes_motor** e os arquivos de log dentro dela:

```
# Criando a pasta e os arquivos "sem querer" como administrador
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sudo mkdir testes_motor
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sudo touch
testes_motor/teste{A..G}.csv
```

Como o comando **sudo** transfere a execução daquele momento para o superusuário, agora tanto a pasta quanto os arquivos pertencem nativamente ao **root**. Você passa a ser obrigado a usar **sudo** para qualquer edição ou exclusão de dados dentro dela.

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l
drwxr-xr-x 2 root root 4096 mar  7 15:39 testes_motor

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l testes_motor/
total 0
-rw-r--r-- 1 root root 0 mar  7 15:43 testeA.csv
-rw-r--r-- 1 root root 0 mar  7 15:43 testeB.csv
-rw-r--r-- 1 root root 0 mar  7 15:43 testeC.csv
-rw-r--r-- 1 root root 0 mar  7 15:43 testeD.csv
-rw-r--r-- 1 root root 0 mar  7 15:43 testeE.csv
-rw-r--r-- 1 root root 0 mar  7 15:43 testeF.csv
-rw-r--r-- 1 root root 0 mar  7 15:43 testeG.csv
```

Para resolver isso da forma correta, usamos o **chown** com **-R** para transferir a posse da pasta e de tudo que estiver dentro dela de uma vez. A estrutura do comando é:

```
sudo chown -R <dono>:<grupo> <pasta>
```

Aplicando ao nosso caso:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sudo chown -R
thallys:thallys testes_motor/
```

Depois, a pasta e todos os arquivos voltam a pertencer ao seu usuário e podem ser editados normalmente sem **sudo**:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l testes_motor/
total 0
-rw-r--r-- 1 thallys thallys 0 mar  7 15:43 testeA.csv
-rw-r--r-- 1 thallys thallys 0 mar  7 15:43 testeB.csv
-rw-r--r-- 1 thallys thallys 0 mar  7 15:43 testeC.csv
-rw-r--r-- 1 thallys thallys 0 mar  7 15:43 testeD.csv
-rw-r--r-- 1 thallys thallys 0 mar  7 15:43 testeE.csv
-rw-r--r-- 1 thallys thallys 0 mar  7 15:43 testeF.csv
-rw-r--r-- 1 thallys thallys 0 mar  7 15:43 testeG.csv
```

Exemplo 2: Transferindo a posse de um arquivo para o root

Um cenário comum no dia a dia é quando um script que você desenvolveu precisa ser transferido para o controle do **root**, pois será executado como parte de um processo do sistema. Vamos transferir o **scripts/main.py** do nosso projeto para o **root**:

Antes o arquivo pertence ao thallys:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l scripts/main.py
-rwxrwxr-- 1 thallys thallys 35 mar  7 14:22 scripts/main.py
```

Transferimos a posse para o root

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sudo chown root:root
scripts/main.py
```

Depois, o arquivo agora pertence ao root:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l scripts/main.py
-rwxrwxr-- 1 root root 35 mar  7 14:22 scripts/main.py
```

A partir desse momento, qualquer tentativa de editar ou mover o **main.py** sem **sudo** será bloqueada pelo sistema. Caso queira reverter e devolver o arquivo ao seu usuário, o caminho é o **chown** novamente, como vimos no exemplo anterior.

Já o **chgrp** é útil quando você quer compartilhar um arquivo ou diretório com um grupo específico sem alterar o dono. Como o **main.py** agora pertence ao **root**, precisamos do **sudo** para alterar qualquer propriedade dele, incluindo o grupo. Vamos transferi-lo para o grupo **thallys** mantendo o **root** como dono:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sudo chgrp thallys
scripts/main.py
```

Resultado: o dono continua sendo o **root**, mas agora o grupo **thallys** também tem acesso ao arquivo conforme as permissões de grupo definidas:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l scripts/main.py
-rwxrwxr-- 1 root thallys 35 mar  7 14:22 scripts/main.py
```

Dominar o **chmod**, o **chown** e o **chgrp** garante controle total sobre quem lê, modifica e executa cada arquivo do seu projeto, evitando acidentes em ambientes compartilhados.

4.4 Permissões Especiais

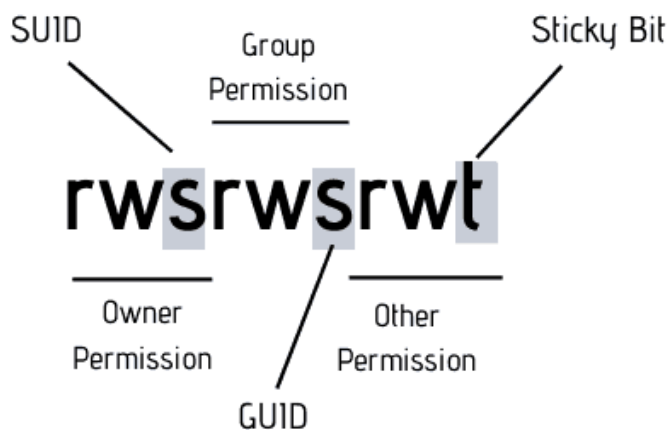


Figura 4: Permissões especiais.

Além das permissões comuns, o Linux possui **permissões especiais** que alteram o comportamento da execução de arquivos e do acesso a diretórios. Elas são representadas por um quarto dígito octal, posicionado à esquerda dos três já conhecidos — por exemplo, **0664**, **4755** ou **1777**.

Assim como as permissões padrão, esses bits especiais possuem valores fixos que ditam o peso de cada um. Na hora de configurar com o **chmod**, você soma esses valores para formar o primeiro dígito: **4 = SUID**, **2 = SGID**, **1 = Sticky bit**.

Valor Octal	Valor Binário (ugo)	Significado
0	000	Nenhuma permissão especial
1	001	Sticky bit
2	010	SGID
3	011	SGID e sticky bit
4	100	SUID
5	101	SUID e sticky bit
6	110	SUID e SGID
7	111	SUID, SGID e sticky bit

- **SUID:** o arquivo é executado com os privilégios do seu dono, não de quem o está rodando. O `chsh` (comando para trocar o shell do usuário) precisa de permissões de `root` para modificar o `/etc/passwd`, então roda com SUID:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l /bin/chsh
-rwsr-xr-x 1 root root 44808 fev  6 2024 /bin/chsh
```

Perceba o `s` no lugar do `x` do dono, é a marca do SUID ativo.

- **SGID:** semelhante ao SUID, mas aplica o grupo do arquivo durante a execução. O `crontab` usa SGID para que usuários comuns possam agendar tarefas sem precisar de `root`:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -l /usr/bin/crontab
-rwxr-sr-x 1 root crontab 39568 mar 23 2022 /usr/bin/crontab
```

Aqui o `s` aparece na posição do grupo, indicando que o processo herda o grupo `crontab` durante a execução.

- **Sticky bit:** aplicado a diretórios, garante que cada usuário só possa apagar seus próprios arquivos, mesmo que todos tenham permissão de escrita. O `/tmp` é o exemplo clássico, e o sticky bit aparece no `t` final ao listar o diretório em si com `ls -ld`:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -ld /tmp
drwxrwxrwt 23 root root 4096 mar  7 15:39 /tmp
```

O `t` no final confirma o sticky bit ativo. Qualquer usuário pode criar arquivos em `/tmp`, mas ninguém pode apagar os arquivos de outro.

Vale notar que nos três casos — SUID, SGID e sticky bit — a string de permissões exibida pelo `ls -l` substitui o `x` de execução por uma letra especial naquela posição. O mesmo caractere `s` aparece tanto na posição do usuário quanto na do grupo porque ele representa o mesmo conceito em contextos diferentes: na posição do usuário indica SUID ativo, na posição do grupo indica SGID ativo. A diferença entre maiúsculo e minúsculo revela se a execução está habilitada ou não junto com a permissão especial:

Caractere	Posição na string	Leitura
<code>s</code>	Usuário	SUID ativo e execução habilitada para o usuário
<code>S</code>	Usuário	SUID ativo, mas sem execução para o usuário
<code>s</code>	Grupo	SGID ativo e execução habilitada para o grupo
<code>S</code>	Grupo	SGID ativo, mas sem execução para o grupo
<code>t</code>	Outros	Sticky bit ativo e execução habilitada para outros

Caractere	Posição na string	Leitura
T	Outros	Sticky bit ativo, mas sem execução para outros